



GIOCO DELL'OCA

Fase di Elaborazione – Iterazione 5

CALOGERO FALCONE & FRANCESCO CATANIA

Sommario

1.0	Introduzione	1
2.0	Caso d'uso UC7: Gestione set di regole predefinite	2
2.1	Caso d'uso UC8: Gestione scenari di gioco	3
2.1	Caso d'uso UC9: Gestione elementi di gioco personalizzabili	4
3.0	Analisi Orientata agli Oggetti	6
3.1	Modello di Dominio	6
3.2	Diagramma di Sequenza di Sistema	8
3.3	Contratti delle operazioni	14
4.0	Progettazione Orientata agli Oggetti	15
4.1	Diagramma di Sequenza	15
4.2	Diagramma delle Classi	18
4.3	Caso d'Uso di Avviamento	18
5.0	Implementazione	19
6.0	Testing	20

1.0 Introduzione

Durante la quinta iterazione verranno analizzati i seguenti requisiti:

- Implementazione dello scenario principale di successo ed estensioni del caso d'uso *UC7: Gestione set di regole predefinite*.
- Implementazione dello scenario principale di successo ed estensioni del caso d'uso *UC8: Gestione scenari di gioco*.
- Implementazione dello scenario principale di successo ed estensioni del caso d'uso *UC9: Gestione elementi di gioco personalizzabili*.
- Arricchimento e perfezionamento dei test unitari dell'applicativo.
- Refactoring della logica di gestione degli effetti delle caselle speciali tramite l'implementazione dei pattern *Singleton* e *Strategy*.
- Affinamento di alcuni dei diagrammi UML introdotti nelle precedenti iterazioni.

2.0 Caso d'uso UC7: Gestione set di regole predefinite

Di seguito viene riportato nel dettaglio il caso d'uso *UC7: Gestione set di regole predefinite*. Questo caso d'uso permette all'amministratore di creare, modificare o eliminare dei set di regole, in modo da renderli disponibili ai giocatori in fase di configurazione della partita.

Caso d'uso	Gestione set di regole predefinite
Nome del caso d'uso	UC7: Gestione set di regole predefinite
Portata	Applicazione Gioco dell'Oca
Livello	Obiettivo Utente
Attore primario	Amministratore
Parti interessate e interessi	Amministratore → Vuole gestire i set di regole predefinite
Pre-condizioni	
Garanzia di successo	Il sistema salva i set di regole impostati dall'amministratore
Scenario principale di successo	1. L'amministratore vuole accedere al pannello admin cliccando il pulsante "Pannello admin". 2. Il sistema apre il popup di accesso al pannello admin. 3. L'amministratore inserisce le credenziali di accesso. 4. Il sistema apre il pannello admin. 5. L'amministratore accede alla pagina di gestione dei set di regole predefinite cliccando il pulsante "Gestione set di regole". 6. Il sistema apre la pagina di gestione dei set di regole predefinite. 7. L'amministratore può aggiungere ed eliminare i set di regole e ha anche la possibilità di modificare le singole regole di ogni set, impostandone il valore. 8. L'amministratore rende effettive le modifiche effettuate cliccando il pulsante "Salva".
Estensioni	3a. L'accesso al pannello degli amministratori fallisce, e il sistema torna al passo 1. 4a. In qualsiasi momento, l'amministratore può cliccare sul pulsante "Torna al menu", che permette di tornare al menu principale.

Requisiti speciali

Elenco delle varianti tecnologiche e dei dati

Frequenza di ripetizioni

Varie

7a. In qualsiasi momento, l'amministratore può tornare al pannello degli admin cliccando il pulsante "Indietro". Se dovessero esserci delle modifiche non salvate, il sistema non ne terrà traccia.

2.1 Caso d'uso UC8: Gestione scenari di gioco

Di seguito viene riportato nel dettaglio il caso d'uso *UC8: Gestione scenari di gioco*. Questo caso d'uso permette all'amministratore di creare, modificare o eliminare gli scenari di gioco, in modo da renderli disponibili ai giocatori in fase di configurazione della partita.

Caso d'uso	Gestione scenari di gioco
Nome del caso d'uso	UC8: Gestione scenari di gioco
Portata	Applicazione Gioco dell'Oca
Livello	Obiettivo Utente
Attore primario	Amministratore
Parti interessate e interessi	Amministratore → Vuole gestire gli scenari
Pre-condizioni	
Garanzia di successo	Il sistema salva gli scenari impostati dall'amministratore
Scenario principale di successo	1. L'amministratore vuole accedere al pannello admin cliccando il pulsante "Pannello admin". 2. Il sistema apre il popup di accesso al pannello admin. 3. L'amministratore inserisce le credenziali di accesso. 4. Il sistema apre il pannello admin. 5. L'amministratore accede alla pagina di gestione degli scenari cliccando il pulsante "Gestione scenari".

Estensioni

Requisiti speciali

Elenco delle varianti tecnologiche e dei dati

Frequenza di ripetizioni

Varie

6. Il sistema apre la pagina di gestione degli scenari.
7. L'amministratore può aggiungere, modificare ed eliminare gli scenari, impostandone anche le varie descrizioni da mostrare per ogni tipologia di casella.
8. L'amministratore rende effettive le modifiche effettuate cliccando il pulsante "Salva".

3a. L'accesso al pannello degli amministratori fallisce, e il sistema torna al passo 1.

4a. In qualsiasi momento, l'amministratore può cliccare sul pulsante "Torna al menu", che permette di tornare al menu principale.

7a. In qualsiasi momento, l'amministratore può tornare al pannello degli admin cliccando il pulsante "Indietro". Se dovessero esserci delle modifiche non salvate, il sistema non ne terrà traccia.

8a. Nel caso in cui l'amministratore non abbia valorizzato, in una o più righe, codice o descrizione dello scenario, il sistema blocca il salvataggio e torna al passo 7.

2.1 Caso d'uso UC9: Gestione elementi di gioco personalizzabili

Di seguito viene riportato nel dettaglio il caso d'uso *UC9: Gestione elementi di gioco personalizzabili*. Questo caso d'uso permette all'amministratore di creare, modificare o eliminare gli elementi di gioco personalizzabili come dadi e pedine, in modo da renderli disponibili ai giocatori in fase di configurazione della partita.

Caso d'uso

Gestione elementi di gioco personalizzabili

Nome del caso d'uso

UC9: Gestione elementi di gioco personalizzabili

Portata

Applicazione Gioco dell'Oca

Livello

Obiettivo Utente

Attore primario	Amministratore
Parti interessate e interessi	Amministratore → Vuole gestire le personalizzazioni
Pre-condizioni	
Garanzia di successo	Il sistema salva le personalizzazioni impostate dall'amministratore
Scenario principale di successo	1. L'amministratore vuole accedere al pannello admin cliccando il pulsante "Pannello admin". 2. Il sistema apre il popup di accesso al pannello admin. 3. L'amministratore inserisce le credenziali di accesso. 4. Il sistema apre il pannello admin. 5. L'amministratore accede alla pagina di gestione delle personalizzazioni cliccando il pulsante "Gestione personalizzazioni". 6. Il sistema apre la pagina di gestione delle personalizzazioni. 7. L'amministratore può aggiungere, modificare ed eliminare i dadi e le pedine, impostandone anche il <i>path</i> contenente l'immagine della singola personalizzazione. 8. L'amministratore rende effettive le modifiche effettuate cliccando il pulsante "Salva".
	3a. L'accesso al pannello degli amministratori fallisce, e il sistema torna al passo 1.
	4a. In qualsiasi momento, l'amministratore può cliccare sul pulsante "Torna al menu", che permette di tornare al menu principale.
Estensioni	7a. In qualsiasi momento, l'amministratore può tornare al pannello degli admin cliccando il pulsante "Indietro". Se dovessero esserci delle modifiche non salvate, il sistema non ne terrà traccia. 8a. Nel caso in cui l'amministratore non abbia valorizzato, in una o più righe, codice o descrizione di una delle personalizzazioni, il sistema blocca il salvataggio e torna al passo 7.
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	
Varie	

3.0 Analisi Orientata agli Oggetti

Anche in questo caso verranno impiegati gli stessi strumenti delle precedenti iterazioni per fornire la descrizione del dominio, ovvero: il *Modello di Dominio*, i *Diagrammi di Sequenza di Sistema* (SSD) e i *Contratti delle Operazioni*.

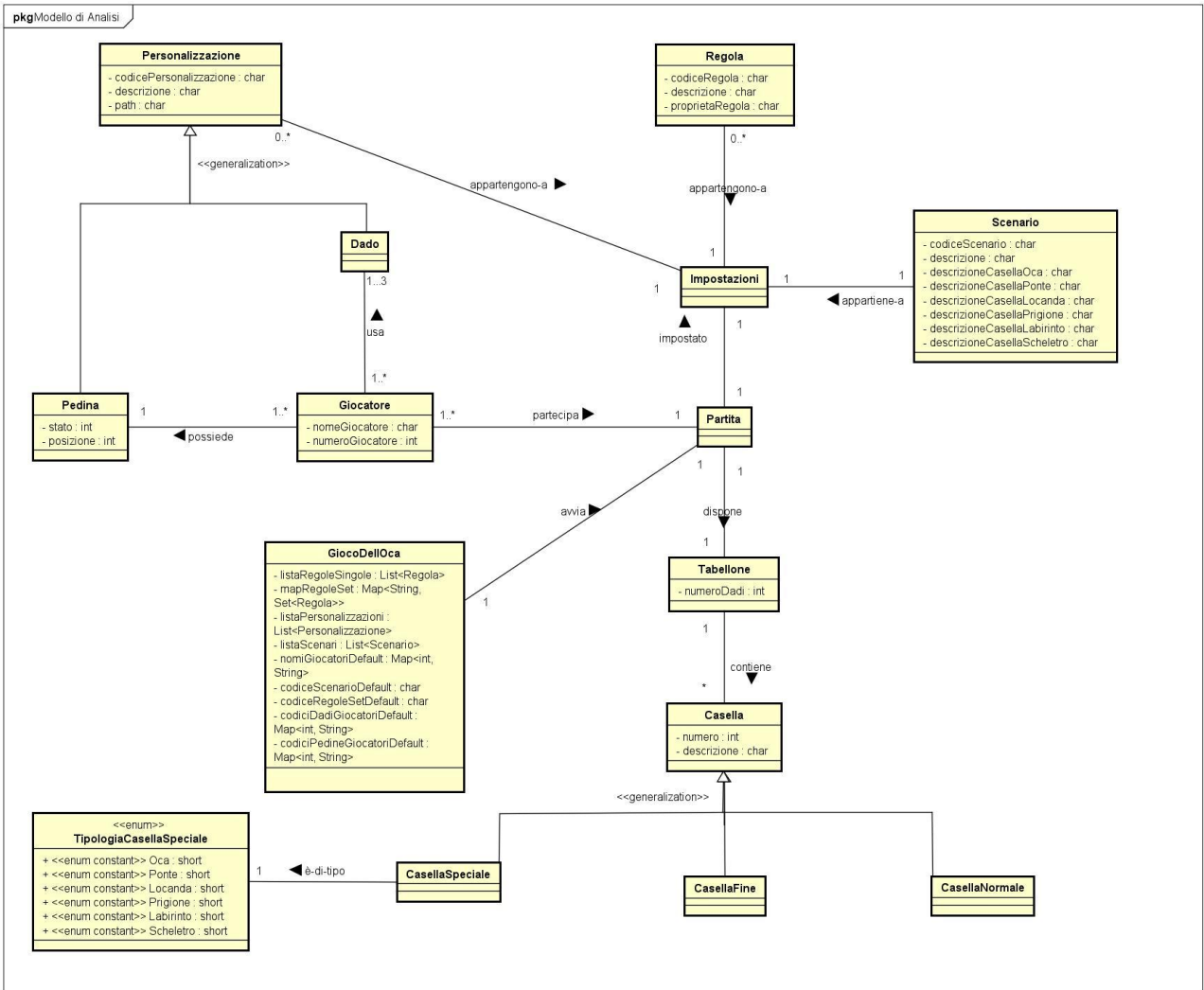
3.1 Modello di Dominio

Bisogna definire il Modello di Dominio, ovvero un elaborato grafico che permette di identificare i concetti, gli attributi e le associazioni più importanti. Dall'analisi dei casi d'uso presi finora in considerazione, valutando lo scenario principale di successo e le rispettive estensioni, è stato possibile identificare le seguenti classi concettuali:

- **Giocatore:** rappresenta l'attore primario, che interagisce con il sistema per eseguire le operazioni, viene definito nell'Iterazione 1. Nell'iterazione 4 sono stati previsti dei campi aggiuntivi per permettere il salvataggio delle impostazioni utente;
- **GiocoDellOca:** rappresenta il sistema che permette la gestione della partita, viene definito nell'Iterazione 1;
- **Partita:** entità che rappresenta la sessione di gioco e che dispone di impostazioni iniziali per il suo funzionamento viene definito nell'Iterazione 1;
- **Impostazioni:** entità che gestisce regole, scenari e personalizzazioni che verranno usati in partita, viene definito nell'Iterazione 1;
- **Scenario:** lo scenario definisce la narrazione che sottintende ogni partita: inciderà dunque sulla descrizione di ogni casella. Viene scelto uno scenario all'inizio di ogni partita, viene definito nell'Iterazione 1;
- **Regola:** le regole rappresentano la modalità di gioco (rapida, normale o lunga) e il numero di dadi che verranno utilizzati dai giocatori nella partita, viene definito nell'Iterazione 1.
- **Personalizzazione:** rappresentano le proprietà di alcuni elementi di gioco sotto il diretto controllo dei giocatori modificabili a piacimento:
 - **Pedina:** rappresenta il Giocatore sulle Caselle del Tabellone. Viene impostato dal Giocatore tramite Impostazioni all'inizio della Partita secondo quanto definito nella Iterazione 1 nel caso di partita singleplayer, e da ciascun giocatore (sia *host* che *guest*) secondo quanto definito nell'Iterazione 3 nel caso di partita multiplayer;
 - **Dado:** oggetto che serve per il movimento della Pedina lungo il Tabellone. Viene impostato dal Giocatore tramite Impostazioni all'inizio della Partita secondo quanto definito nella Iterazione 1, e da ciascun giocatore (sia *host* che *guest*) secondo quanto definito nell'Iterazione 3 nel caso di partita multiplayer;

- **Tabellone:** si tratta del campo di gioco messo a disposizione dal controller GiocoDell'Oca tramite Partita. L'interazione con l'oggetto Tabellone non è diretta, ma passa attraverso le Caselle che lo costituiscono;
- **Casella:** rappresentano le varie posizioni che il Giocatore tramite la Pedina può assumere. La Casella può essere di tre tipi: **Casella Normale**, **Casella Speciale** e **Casella Fine**. A sua volta, sono presenti diversi **tipi di Casella Speciale**: Oca, Ponte, Locanda, Prigione, Labirinto e Scheletro.

Tenendo conto di associazioni e attributi, si costruisce il seguente Modello di Dominio:



3.2 Diagramma di Sequenza di Sistema

Procedendo con la **OOA**, si passa ora alla creazione del Diagramma di Sequenza di Sistema (SSD) per poter descrivere il corso degli eventi di I/O per lo scenario principale di successo dei casi d'uso trattati.

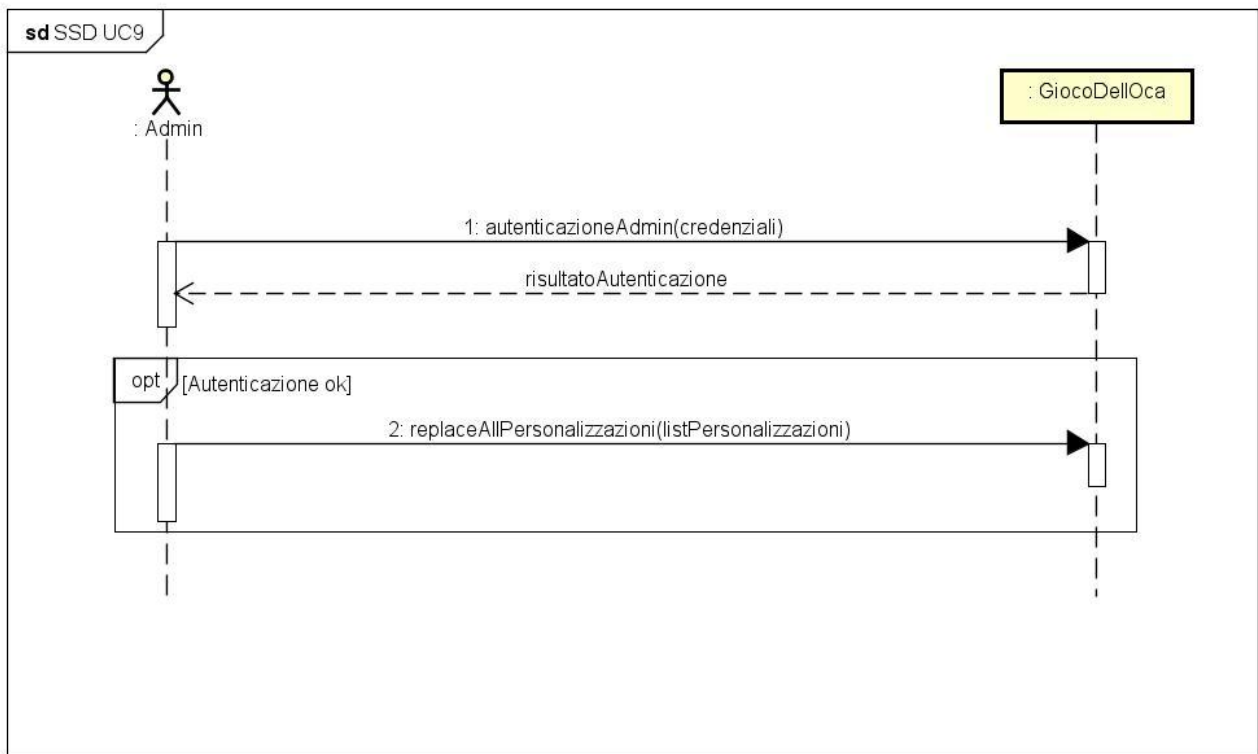
Il diagramma dell'*UC7: Gestione set di regole predefinite* sarà il seguente:



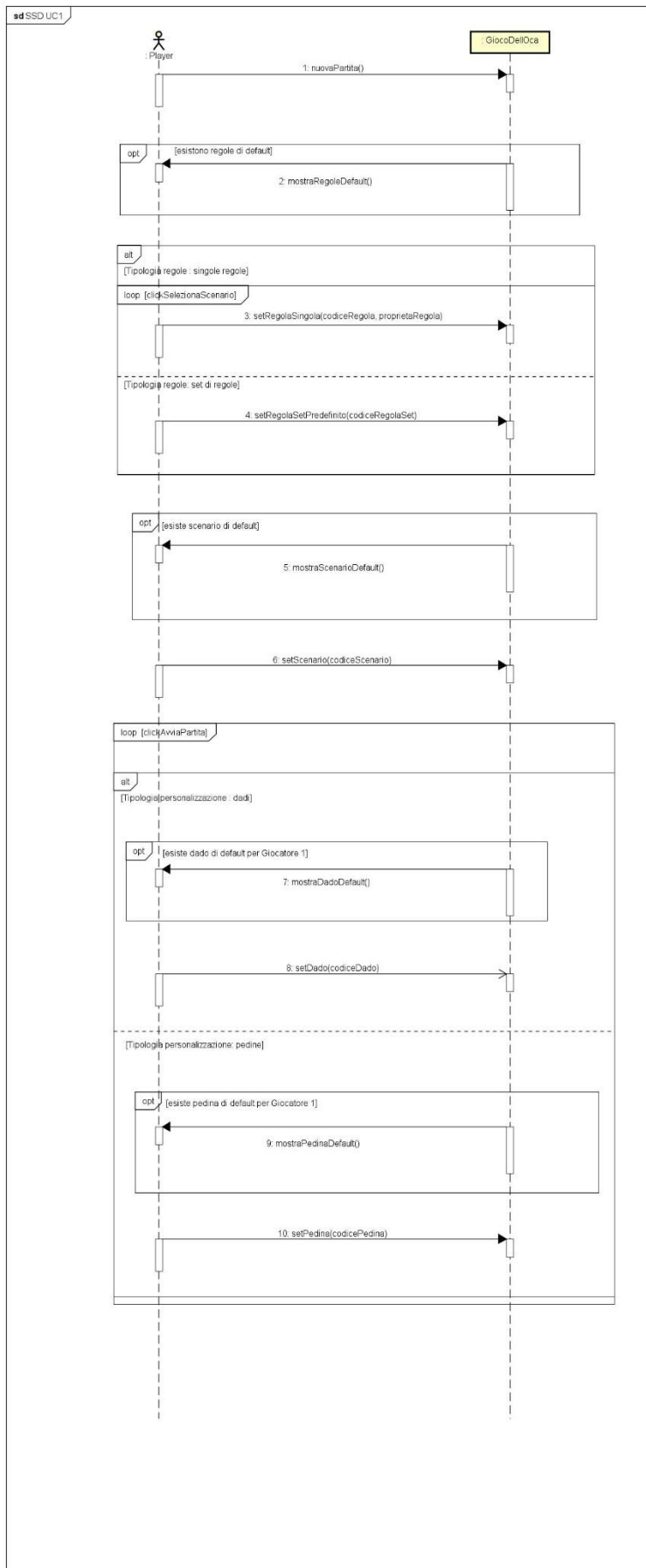
Il diagramma dell'*UC8: Gestione scenari di gioco* sarà il seguente:

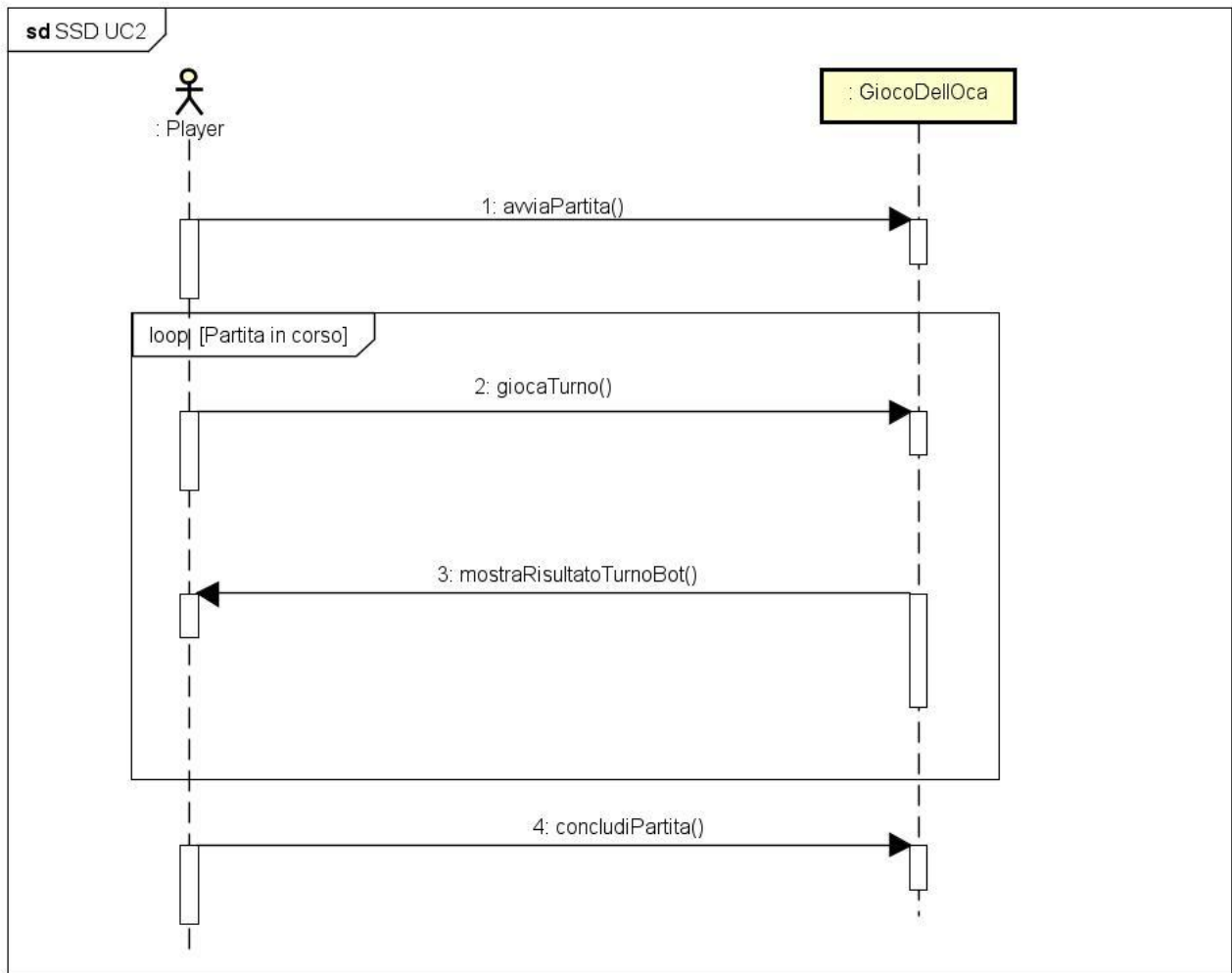


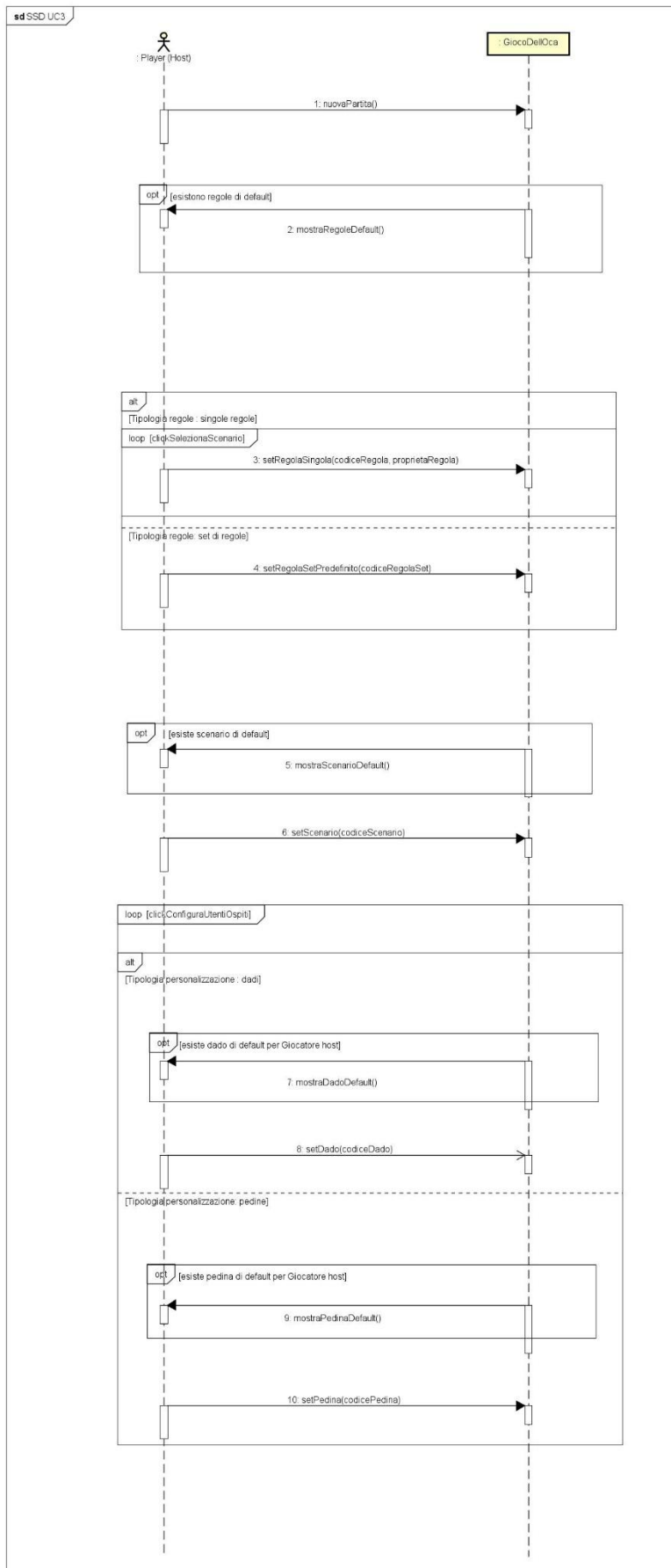
Il diagramma dell'UC9: *Gestione elementi di gioco personalizzabili* sarà il seguente:

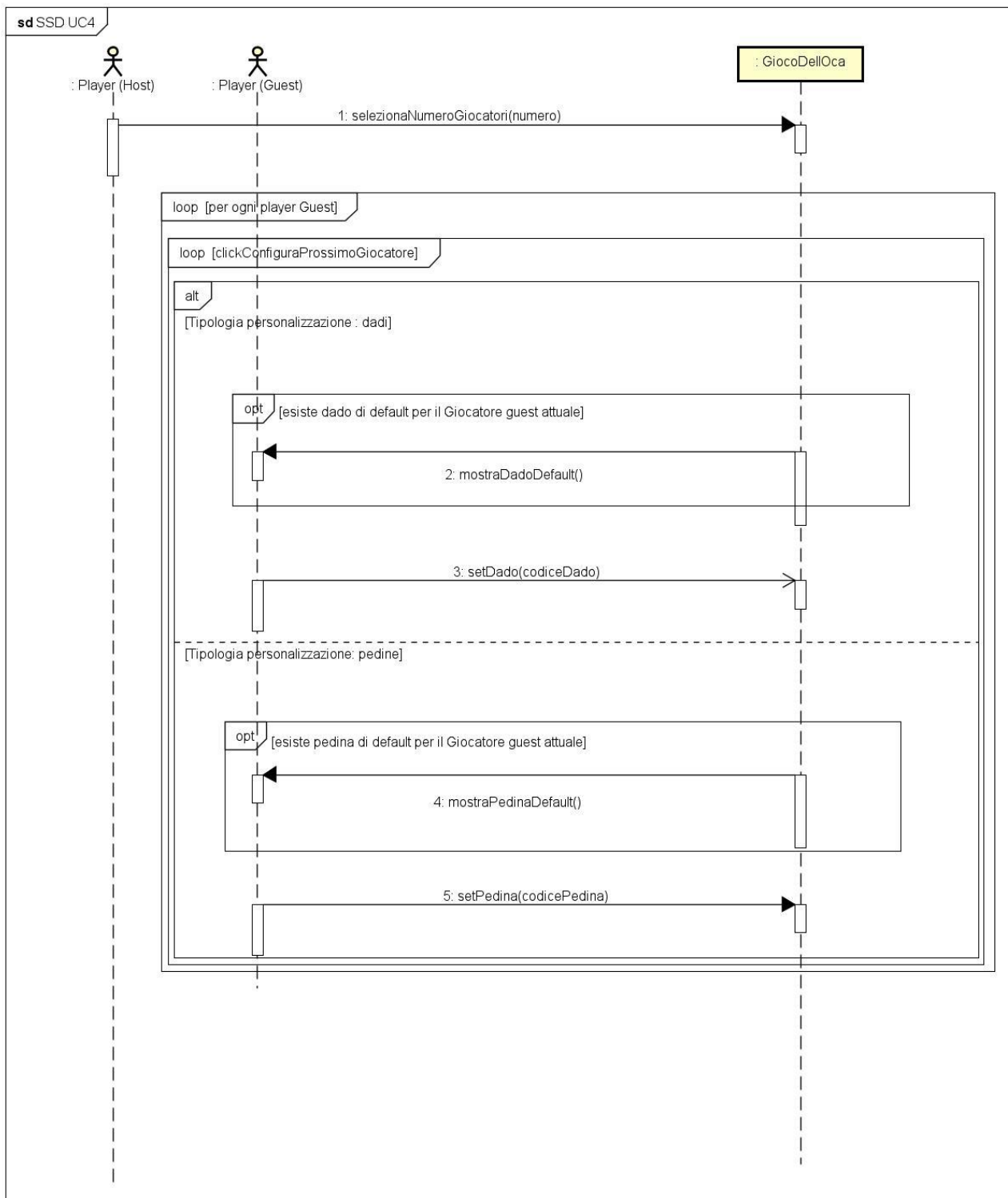


Di seguito viene mostrato l'affinamento di alcuni SSD introdotti nelle precedenti iterazioni:









3.3 Contratti delle operazioni

Attraverso l'utilizzo dei Contratti delle Operazioni, verranno descritte le principali operazioni di sistema responsabili della gestione degli eventi rilevati e analizzati.

Contratto C01 *autenticazioneAdmin*

Operazioni	autenticazioneAdmin(credenziali)
Riferimenti	Casi d'uso UC7, UC8 e UC9
Pre-condizioni	
Post-condizioni	L'amministratore viene autenticato

Contratto C02 *replaceAllSetRegole*

Operazioni	replaceAllSetRegole(mapRegoleSet)
Riferimenti	Caso d'uso UC7: Gestione set di regole predefinite
Pre-condizioni	L'amministratore è stato autenticato con successo
Post-condizioni	Sono stati salvati nel sistema i nuovi set di regole precedentemente impostati

Contratto C03 *replaceAllScenari*

Operazioni	replaceAllScenari(listScenari)
Riferimenti	Caso d'uso UC8: Gestione scenari di gioco
Pre-condizioni	L'amministratore è stato autenticato con successo
Post-condizioni	Sono stati salvati nel sistema i nuovi scenari precedentemente impostati

Contratto C04 *replaceAllPersonalizzazioni*

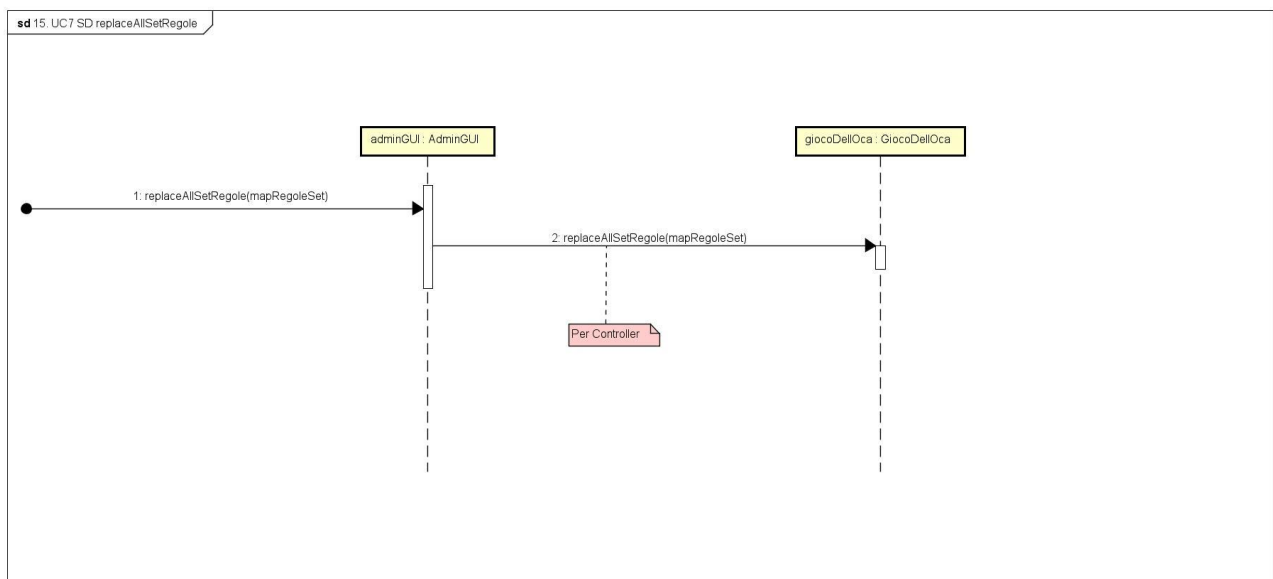
Operazioni	replaceAllPersonalizzazioni(listPersonalizzazioni)
Riferimenti	Caso d'uso UC9: Gestione elementi di gioco personalizzabili
Pre-condizioni	L'amministratore è stato autenticato con successo
Post-condizioni	Sono stati salvati nel sistema le nuove personalizzazioni (dadi e pedine) precedentemente impostate

4.0 Progettazione Orientata agli Oggetti

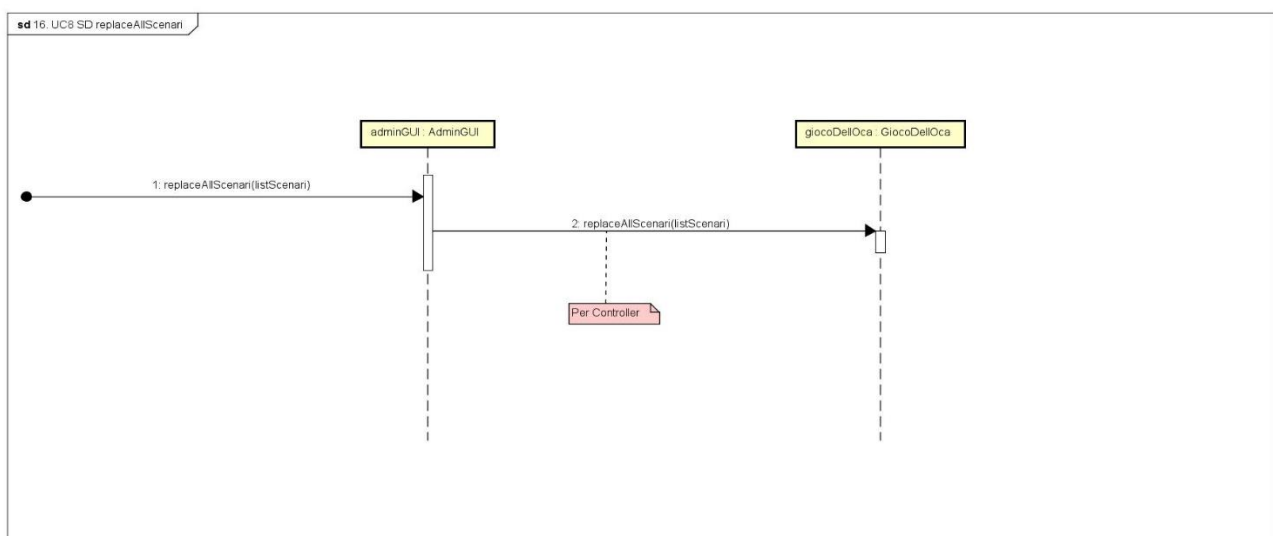
Di seguito, in relazione al Modello di Progetto, vengono presentati i diagrammi di sequenza riguardanti i casi d'uso *UC7: Gestione set di regole predefinite*, *UC8: Gestione scenari di gioco* e *UC9: Gestione elementi di gioco personalizzabili*.

4.1 Diagramma di Sequenza

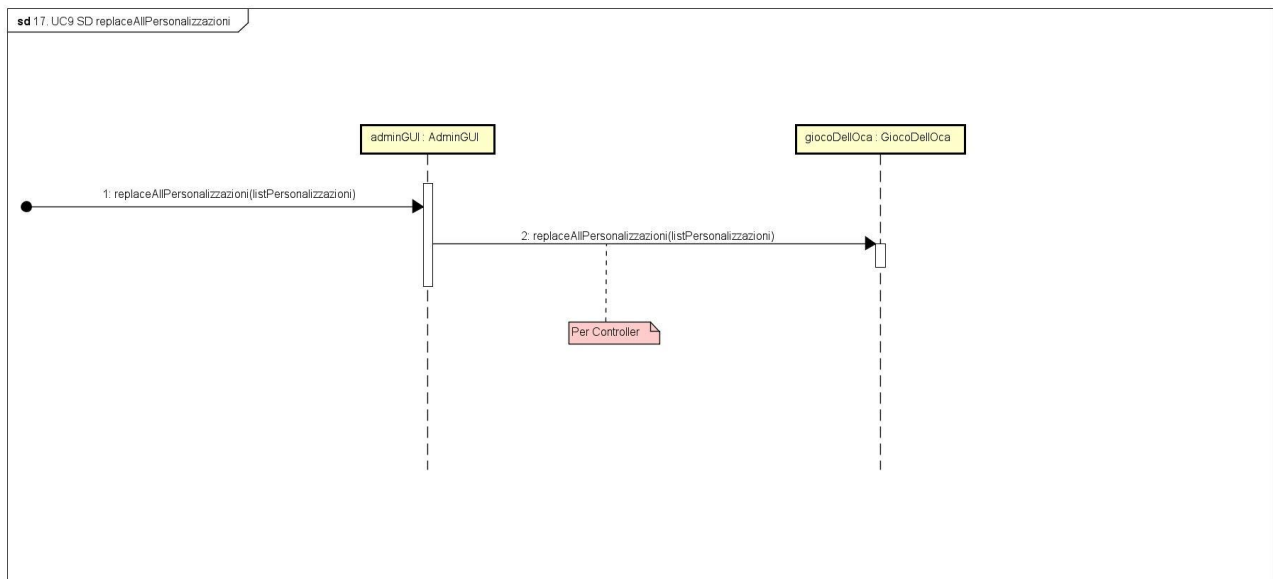
- SD replaceAllSetRegole



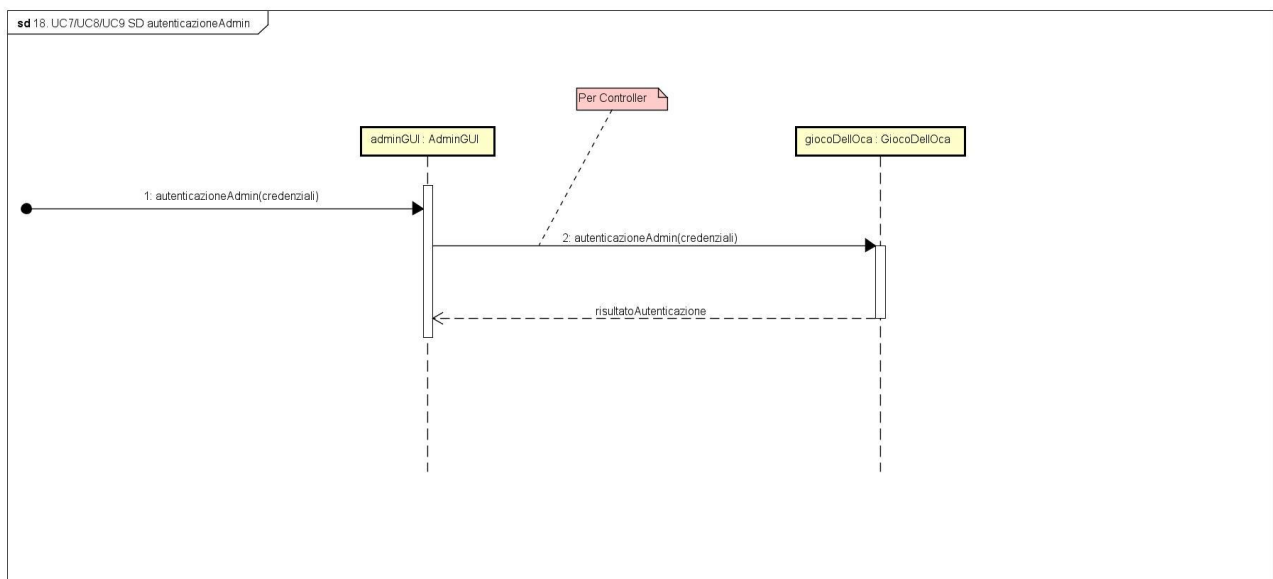
- SD replaceAllScenari



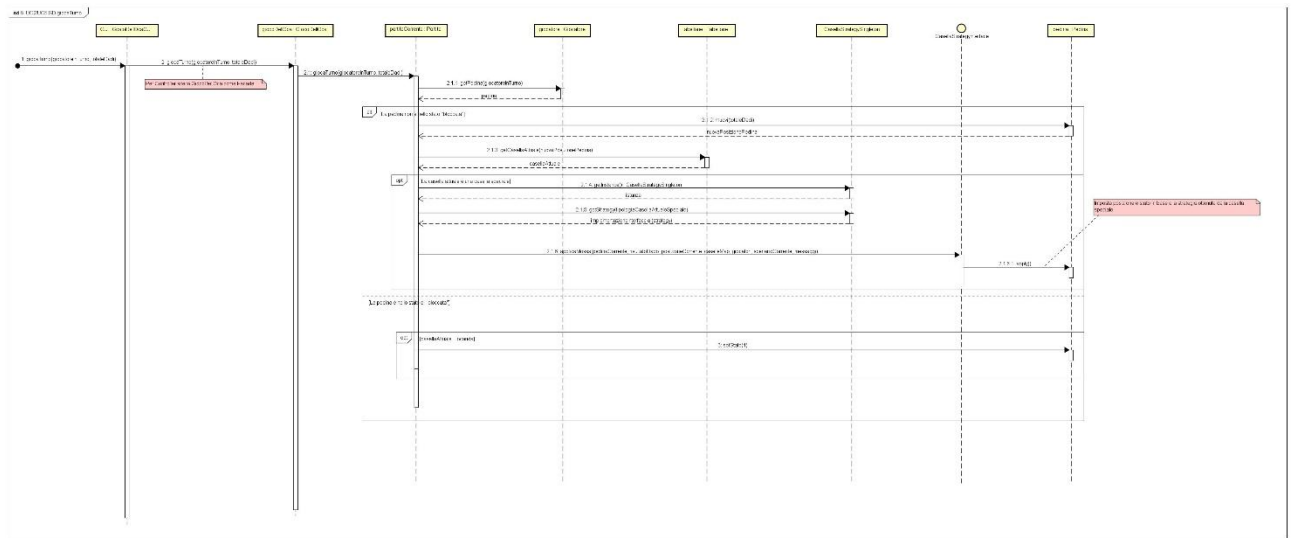
- SD replaceAllPersonalizzazioni



- SD autenticazioneAdmin



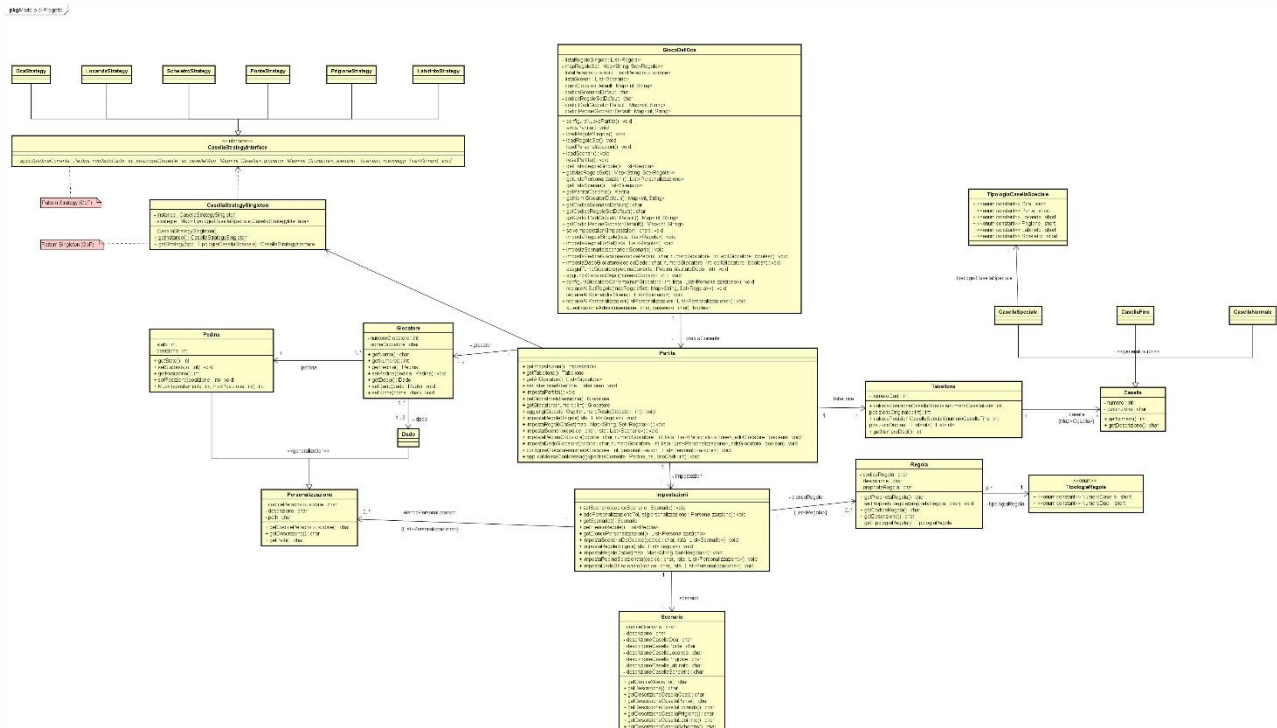
- Refactoring del SD giocaTurno, introdotto nelle precedenti iterazioni, applicando i pattern *Singleton* e *Strategy*:



4.2 Diagramma delle Classi

Sono state introdotte, in questa iterazione, l'interfaccia e le classi necessarie per la gestione degli effetti delle caselle speciali, secondo i pattern *Singleton* e *Strategy*. La classe **Partita** recupererà la strategia associata alla casella speciale richiesta, interfacciandosi con un oggetto *Singleton*. Tale oggetto, con un punto di accesso globale, consente, appunto, di individuare una delle *Strategy* che implementano l'interfaccia comune, in base alla casella speciale che gli viene passata in input.

Di seguito il diagramma delle classi completo della modifica sopracitata:



4.3 Caso d'Uso di Avviamento

Il Caso d'Uso di Avviamento non presenta modifiche rispetto a quello presentato durante l'Iterazione 1.

5.0 Implementazione

Per la quinta e ultima iterazione sono state seguite le stesse scelte progettuali e implementative delle iterazioni precedenti, ovvero:

- Si è scelto di non utilizzare un database per memorizzare i dati in maniera permanente, ma di mantenerli in memoria principale;
- È stata implementata una semplice interfaccia tramite Java Swing.
- Per semplicità, le credenziali di accesso al pannello degli admin sono:
username -> "admin", password -> "admin".
Una gestione più articolata delle credenziali degli amministratori potrebbe essere oggetto di eventuali evolutive future del software.

6.0 Testing

I test sono stati eseguiti sulla seguente classe:

- `GiocoDellOcaTest`

I metodi testati sono i seguenti metodi provenienti dalla classe `GiocoDellOca`:

1. `void impostazioniTest()`
2. `void impostaRegoleSingoleTest()`
3. `void impostaRegoleDaSetTest()`
4. `void impostaScenarioTest()`
5. `void impostaPedinaTest()`
6. `void impostaDadoTest()`
7. `void eseguiTurnoTest()`
8. `void aggiungiGiocatoriOspitiTest()`
9. `void configuraGiocatoreCorrenteTest()`
10. `void autenticazioneAdminTest()`
11. `void replaceAllScenariTest()`
12. `void replaceAllRegoleTest()`
13. `void replaceAllPersonalizzazioniTest()`

`void impostazioniTest()`

Test del salvataggio delle impostazioni per avviare il gioco.

Condizione iniziale: che esista una variabile `giocoDellOca`.

Condizione da testare: che esistano i giocatori, le personalizzazioni, lo scenario e le regole impostate.

`void impostaRegoleSingoleTest()`

Test dell'impostazione di una regola singola.

Condizione iniziale: che esista una variabile `giocoDellOca` e che si stia configurando una nuova partita.

Condizione da testare: perché il metodo funzioni si deve verificare che il numero di regole che vengono salvate nelle impostazioni sia 0 se la tabella è vuota (non sono state aggiunte manualmente via GUI) oppure uguale al numero di record passato dalla tabella se quest'ultima è stata valorizzata.

void impostaRegoleDaSetTest()

Test dell'impostazione di un set di regole.

Condizione iniziale: che esista una variabile `giocoDellOca` e che si stia configurando una nuova partita.

Condizione da testare: perché il metodo funzioni si deve verificare che il numero di regole che vengono salvate nelle impostazioni sia 0 se la tabella è vuota (non sono state aggiunte manualmente via GUI) oppure uguale al numero di record passato dalla tabella se quest'ultima è stata valorizzata.

void impostaScenarioTest()

Test dell'impostazione di uno scenario.

Condizione iniziale: che esista una variabile `giocoDellOca` e che si stia configurando una nuova partita.

Condizione da testare: perché il metodo funzioni si deve verificare che lo scenario scelto esista tra quelli definiti nella lista di `giocoDellOca` altrimenti viene impostato lo scenario di default.

void impostaPedinaTest()

Test dell'impostazione di una pedina abbinata ad un giocatore.

Condizione iniziale: che esista una variabile `giocoDellOca`, che si stia configurando una nuova Partita e che esista un giocatore per poter associare la pedina.

Condizione da testare: perché il metodo funzioni si deve verificare che la pedina scelta esista e sia stata associata correttamente al giocatore.

void impostaDadoTest()

Test dell'impostazione di un dado abbinato ad un giocatore.

Condizione iniziale: che esista una variabile `giocoDellOca`, che si stia configurando una nuova Partita e che esista un giocatore per poter associare il dado.

Condizione da testare: perché il metodo funzioni si deve verificare che il dado scelto esista e sia stata associato correttamente al giocatore.

void eseguiTurnoTest()

Test dell'esecuzione di un turno di un giocatore.

Condizione iniziale: che esista una variabile `giocoDellOca`, che sia stata configurata e avviata una partita con delle impostazioni di default, e che esista una pedina.

Condizione da testare: perché il metodo funzioni si deve verificare che per ogni possibile tipologia di casella sulla quale la pedina può atterrare, la posizione e lo stato di quest'ultima siano corrette.

void aggiungiGiocatoriOspitiTest()

Test dell'aggiunta giocatori ospiti.

Condizione iniziale: che esista una variabile `giocoDellOca` e che si stia configurando una partita.

Condizione da testare: perché il metodo funzioni si deve verificare che il numero di giocatori aggiunti corrisponda effettivamente al numero di giocatori registrato per la partita corrente.

void configuraGiocatoreCorrenteTest()

Test della configurazione giocatore corrente da fare muovere lungo il tabellone.

Condizione iniziale: che esista una variabile `giocoDellOca` e che stia configurando/sia già stata configurata una nuova partita.

Condizione da testare: perché il metodo funzioni si deve verificare che, qualora le tabelle preposte alla definizione di pedina e dado siano vuote, vengano associate delle personalizzazioni di default, e il giocatore viene impostato come giocatore corrente.

void autenticazioneAdminTest()

Test della autenticazione come admin per poter definire le operazioni

Condizione iniziale: che esista una variabile `giocoDellOca` e che si voglia accedere al pannello amministratore.

Condizione da testare: perché il metodo funzioni si deve passare in input la coppia corretta username e password "admin" "admin", altrimenti viene restituito un booleano che blocca la funzionalità.

void replaceAllScenariTest()

Test rimpiazzo scenari di default con scenari nuovi.

Condizione iniziale: che esista una variabile `giocoDellOca` e che gli scenari vecchi esistano già per poter fare il confronto.

Condizione da testare: perché il metodo funzioni si deve verificare che i nuovi elementi siano effettivamente restituiti dalla `getListaScenari` di `giocoDellOca` e siano diversi dai vecchi.

void replaceAllRegoleTest()

Test rimpiazzo set regole di default con regole nuovi.

Condizione iniziale: che esista una variabile `giocoDellOca` e che i set di regole vecchi esistano già per poter fare il confronto.

Condizione da testare: perché il metodo funzioni si deve verificare che i nuovi elementi siano effettivamente restituiti dalla `getMapRegoleSet` di `giocoDellOca` e siano diversi dai vecchi.

void replaceAllPersonalizzazioniTest()

Test rimpiazzo personalizzazioni di default con personalizzazioni nuove.

Condizione iniziale: che esista una variabile `giocoDellOca` e che le personalizzazioni vecchie esistano già per poter fare il confronto.

Condizione da testare: perché il metodo funzioni si deve verificare che i nuovi elementi siano effettivamente restituiti dalla `getListaPersonalizzazioni` di `giocoDellOca` e siano diversi dai vecchi.